

The Linux Concept Journey

version 2.0

July-2023

By Dr. Shlomi Boutnaru



Created using [Craiyon AI Image Generator](#)

Introduction	3
The Auxiliary Vector (AUXV)	4
command not found	5
Out-of-Memory Killer (OOM killer)	6
Why doesn't "ltrace" work on new versions of Ubuntu?	7
vsyscall (Virtual System Calls)	9
vDSO (Virtual Dynamic Shared Object)	10
Calling syscalls from Python	12
Syscalls' Naming Rule: What if a syscall's name starts with "f"?	13
Syscalls' Naming Rule: What if a syscall's name starts with "l"?	14
RCU (Read Copy Update)	15
cgroups (Control Groups)	17
Package Managers	18
What is an ELF (Executable and Linkable Format) ?	19
The ELF (Executable and Linkable Format) Header	20
File System Hierarchy in Linux	21
What is an inode?	23
Why is removing a file not dependent on the file's permissions?	24
VFS (Virtual File System)	25

Introduction

When starting to learn Linux I believe that they are basic concepts that everyone needs to know about. Because of that I have decided to write a series of short writeups aimed at providing the basic vocabulary and understanding for achieving that.

Overall, I wanted to create something that will improve the overall knowledge of Linux in writeups that can be read in 1-3 mins. I hope you are going to enjoy the ride.

Lastly, you can follow me on twitter - @boutnaru (<https://twitter.com/boutnaru>). Also, you can read my other writeups on medium - <https://medium.com/@boutnaru>.

Lets GO!!!!!!

The Auxiliary Vector (AUXV)

There are specific OS variables that a program would probably want to query such as the size of a page (part of memory management - for a future writeup). So how can it be done?

When the OS executes a program it exposes information about the environment in a key-value data store called “auxiliary vector” (in short auxv/AUXV). If we want to check which keys are available we can go over¹ (on older versions it was part of elf.h) and look for all the defines that start with “AT_”.

Among the information that is included in AUXV we can find: the effective uid of the program, the real uid of the program, the system page size, number of program headers (of the ELF - more on that in the future), minimal stack size for signal delivery (and there is more).

If we want to see all the info of AUXV while running a program we can set the LD_AUXV_SHOW environment variable to 1 and execute the requested program (see the screenshot below, it was taken from JSLinux running Fedora 33 based on a riscv64 CPU². We can see that the name of the variable starts with “LD_”, it is because it is used/parsed by the dynamic linker/loader (aka ld.so).

Thus, if we statically link our program (like using the -static flag on gcc) setting the variable won't print the values of AUXV. Anyhow, we can also access the values in AUXV using the “unsigned long getauxval(unsigned long type)” library function³. A nice fact is that the auxiliary vector is located next to the environment variables check the following illustration⁴.

```
[root@localhost ~]# export LD_SHOW_AUXV=1
[root@localhost ~]# uname -a
AT_SYSINFO_EHDR: 0x200001f000
AT_HWCAP: 112d
AT_PAGESZ: 4096
AT_CLKTCK: 100
AT_PHDR: 0x2aaaaaa040
AT_PHENT: 56
AT_PHNUM: 9
AT_BASE: 0x2000000000
AT_FLAGS: 0x0
AT_ENTRY: 0x2aaaaabd38
AT_UID: 0
AT_EUID: 0
AT_GID: 0
AT_EGID: 0
AT_SECURE: 0
AT_RANDOM: 0x3ffffff6821
AT_EXECFN: /bin/uname
Linux localhost 4.15.0-00049-ga3b1e7a-dirty #11 Thu Nov 8 20:30:26 CET 2018 riscv64 riscv64 riscv64 GNU/Linux
```

¹ <https://elixir.bootlin.com/linux/latest/source/include/uapi/linux/auxvec.h#L10>

² <https://bellard.org/jslinux/>

³ <https://man7.org/linux/man-pages/man3/getauxval.3.html>

⁴ <https://static.lwn.net/images/2012/auxvec.png>

command not found

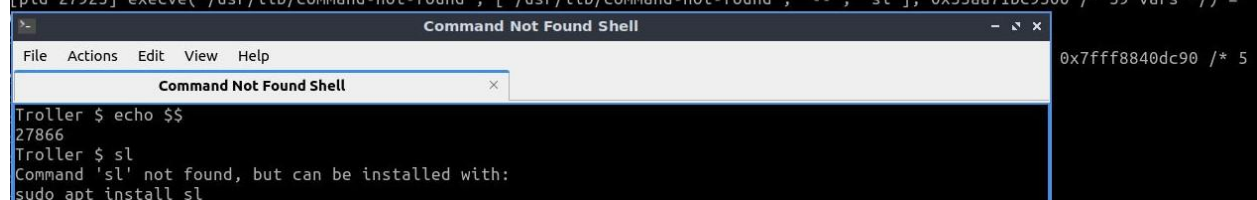
Have you ever asked yourself what happens when you see “command not found” on bash? This writeup is not going to talk about that and not about the flow which determines if a command is found or not (that is a topic for a different write up ;-).

I am going to focus my discussion on what happens in an environment based on bash + Ubuntu (version 22.04). I guess you at least once wrote “sl” instead of “ls” and you got a message “Command 'sl' not found, but can be installed with: sudo apt install sl” - how did bash know that there is such a package that could be installed? - as shown in the screenshot below

Overall, the magic happens with the python script “/usr/lib/command-not-found” which is executed when a bash does not find a command - as shown in the screenshot below. This feature is based on an sqlite database that has a connection between command and packages, it is sorted in “/var/lib/command-not-found/commands.db”.

Lastly, there is a nice website <https://command-not-found.com/> which allows you to search for a command and get a list of different ways of installing it (for different Linux distributions/Windows/MacOS/Docker/etc).

```
Troller$ sudo strace -f -e execve -p 27866
strace: Process 27866 attached
strace: Process 27924 attached
strace: Process 27925 attached
[pid 27925] execve("/usr/lib/command-not-found", ["/usr/lib/command-not-found", "-.", "sl"], 0x55aa71bc9300 /* 59 vars */) =
0x7fff8840dc90 /* 5
```



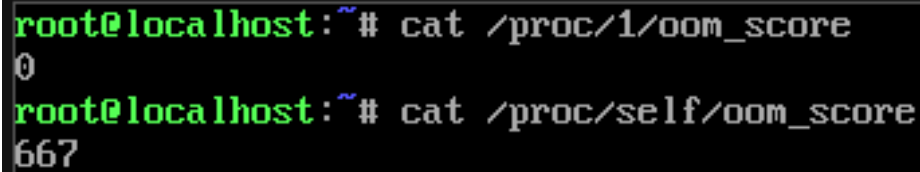
The screenshot shows a terminal window titled "Command Not Found Shell". The terminal output displays the command "sl" being executed, followed by the message "Command 'sl' not found, but can be installed with: sudo apt install sl". The terminal also shows the output of "echo \$\$" which is "27866".

Out-of-Memory Killer (OOM killer)

The Linux kernel has a mechanism called “out-of-memory killer” (aka OOM killer) which is used to recover memory on a system. The OOM killer allows killing a single task (called also oom victim) while that task will terminate in a reasonable time and thus free up memory.

When OOM killer does its job we can find indications about that by searching the logs (like /var/log/messages and grepping for “Killed”). If you want to configure the “OOM killer”⁵.

It is important to understand that the OOM killer chooses between processes based on the “oom_score”. If you want to see the value for a specific process we can just read “/proc/[PID]/oom_score” - as shown in the screenshot below. If we want to alter the score we can do it using “/proc/[PID]/oom_score_adj” - as shown also in the screenshot below. The valid range is from 0 (never kill) to 1000 (always kill), the lower the value is the lower is the probability the process will be killed⁶.



```
root@localhost:~ # cat /proc/1/oom_score
0
root@localhost:~ # cat /proc/self/oom_score
667
```

⁵ <https://www.oracle.com/technical-resources/articles/it-infrastructure/dev-oom-killer.html>

⁶ <https://man7.org/linux/man-pages/man5/proc.5.html>

Why doesn't "ltrace" work on new versions of Ubuntu?

Many folks have asked me about that, so I have decided to write a short answer about it. Two well known command line tools on Linux which can help with dynamic analysis are "strace" and "ltrace". "strace" allows tracing of system calls ("man 2 syscalls") and signals ("man 7 signal"). I am not going to focus on "strace" in this writeup, you can read more about it using "man strace".

On the other hand, "ltrace" allows the tracing of dynamic library calls and signals received by the traced process ("man 1 ltrace"). Moreover, it can also trace syscalls (like "strace") if you are using the "-S" flag.

If you have tried using "ltrace" in the new versions of Ubuntu you probably saw that the library calls are not shown (you can verify it using "ltrace `which ls`"). In order to demonstrate that I have created a small c program - as you can see in the screenshot below ("code.c").

First, if we compile "code.c" and run it using "ltrace" we don't get any information about a library call (see in the screenshot below). Second, if we compile "code.c" with "-z lazy" we can see when running the executable with "ltrace" we do get information about the library functions. So what is the difference between the two?

"ltrace" (and "strace") works by inserting a breakpoint in the PLT for the relevant symbol (that is library function) we want to trace. I have explained about breakpoints in a previous writeup that you can read in the following like - <https://medium.com/@boutnaru/have-you-ever-asked-yourself-how-breakpoints-work-c72dd8619538>. So because by default the binaries are not compiled with "lazy loading" of symbols they are resolved when the application starts and thus the breakpoints set by "ltrace" are not triggered (and we don't see any library calls in the output - as shown in the screenshot below). Also, you can read more about the internals of "ltrace" here - <https://www.kernel.org/doc/ols/2007/ols2007v1-pages-41-52.pdf>

```

Troller # cat code.c
#include <stdio.h>

void main()
{
    printf("\n*****!!!!!!!\n");
}
Troller # gcc code.c -o code
Troller # ./code

*****!!!!!!!
Troller # ltrace ./code

*****!!!!!!!
+++ exited (status 24) +++
Troller # gcc -z lazy code.c -o ./code
Troller # ./code

*****!!!!!!!
Troller # ltrace ./code
puts("\n*****!!!!!!!")
*****!!!!!!!
)                                     = 24
+++ exited (status 24) +++
Troller # █

```


vsyscall (Virtual System Calls)

vsyscall is an old mechanism (replaced by vDSO) which allows the Linux kernel to accelerate the execution of specific system calls. (<https://lwn.net/Articles/446528/>)

vDSO (Virtual Dynamic Shared Object)

vDSO is a shared library that the kernel maps into the memory address space of every user-mode application. It is not something that developers need to think about due to the fact it is used by the C library⁷.

Overall, the reason for even having vDSO is the fact they are specific system calls that are used frequently by user-mode applications. Due to the time/cost of the context-switching between user-mode and kernel-mode in order to execute a syscall it might impact the overall performance of an application.

Thus, vDSO provides “virtual syscalls” due to the need for optimizing system calls implementations. The solution needed to not require libc to track CPU capabilities and or the kernel version. Let us take for example x86, which has two ways of invoking a syscall: “int 0x80” or “sysenter”. The “sysenter” option is faster, due to the fact we don’t need to through the IDT (Interrupt Descriptor Table). The problem is it is supported for CPUs newer than Pentium II and for kernel versions greater than 2.6⁸.

If you vDSO the implementation of the syscall interface is defined by the kernel in the following manner. A set of CPU instructions formatted as ELF is mapped to the end of the user-mode address space of all processes - as shown in the screenshot below. When libc needs to execute a syscall it checks for the presence of vDSO and if it is relevant for the specific syscalls the implementation in vDSO is going to be used - as shown in the screenshot below⁹.

Moreover, for the case of “virtual syscalls” (which are also part of vDSO) there is a frame mapped as two different pages. One in kernel space which is static/”readable & writeable” and the second one in user space which is marked as “read-only”. Two examples for that are the syscalls “getpid()” (which is a static data example) and “gettimeofday()” (which is a dynamic read-write example).

Also, as part of the kernel compilation process the vDSO code is compiled and linked. We can most of the time find it using the following command “find arch/\$ARCH/ -name '*vdso*.so*' -o -name '*gate*.so*'”¹⁰

If we want to enable/disable vDSO we can set “/proc/sys/vm/vdso_enable” to 1/0 respectively¹¹. Lastly, a benchmark of different syscalls using different implementations is shown below.

⁷ <https://man7.org/linux/man-pages/man7/vdso.7.html>

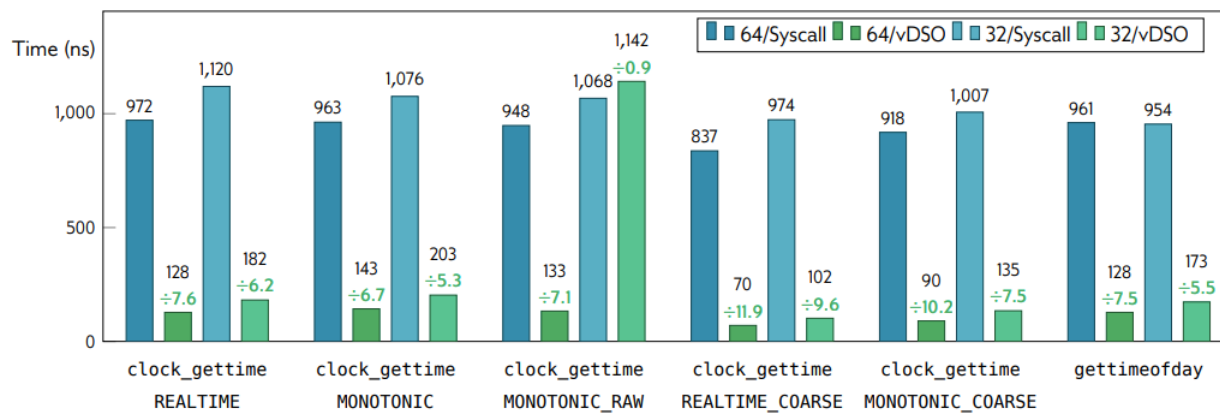
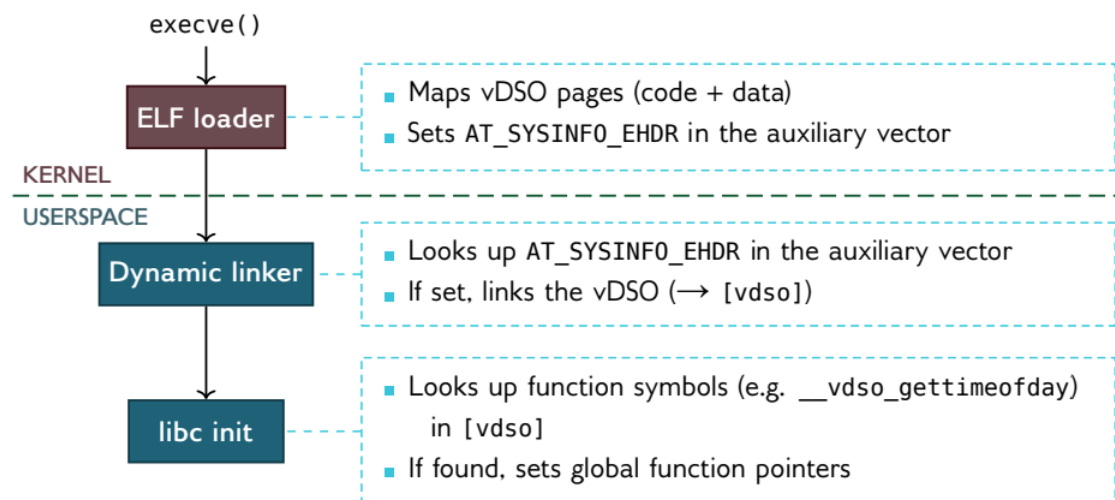
⁸ <https://linux-kernel-labs.github.io/refs/heads/master/so2/lec2-syscalls.html>

⁹ <https://hackmd.io/@sysprog/linux-vdso>

¹⁰ <https://manpages.ubuntu.com/manpages/xenial/man7/vdso.7.html>

¹¹ <https://talk.maemo.org/showthread.php?t=32696>

Kernel and userspace setup



Calling syscalls from Python

Have you ever wanted a quick way to call a syscall (even if it is not exposed by libc)? There is a quick way of doing that using “ctypes” in Python.

We can do it using the “syscall” exported function by libc (check out ‘man 2 syscall’¹² for more information). By calling that function we can call any syscall by passing its number and parameters.

How do we know what the number of the syscall is? We can just check <https://filippo.io/linux-syscall-table/>. What about the parameters? We can just go to the source code which is pointed in any entry of a syscall (from the previous link) or we can just use man (using the following pattern - ‘man 2 {NameOfSyscall}’, for example ‘man 2 getpid’).

Let us see an example, we will use the syscall getpid(), which does not get any arguments. Also, the number of the syscall is 39 (on x64 Linux). You can check the screenshot below for the full example. By the way, the example was made with https://www.tutorialspoint.com/linux_terminal_online.php and online Linux terminal (kernel 3.10).

```
$ python3
Python 3.8.6 (default, Jan 29 2021, 17:38:16)
[GCC 8.4.1 20200928 (Red Hat 8.4.1-1)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import ctypes
>>> libc=ctypes.CDLL(None)
>>> libc.syscall(39)
47617
```

¹² <https://man7.org/linux/man-pages/man2/syscall.2.html>

Syscalls' Naming Rule: What if a syscall's name starts with "f"?

Due to the large number of syscalls, there are some naming rules used in order to help in understanding the operation performed by each of them. Let me go over some of them to give more clarity.

If we have a syscall "<syscall_name>" so we could also have "f<syscall_name>" which means that "f<syscall_name>" does the same operation as "<syscall_name>" but on a file referenced by an fd (file descriptor). Some examples are ("chown", "fchown") and ("stat", "fstat"). It is important to understand that not every syscall which starts with "f" is part of such a pair, look at "fsync()" as an example, however in this case the prefix still denoting the input of the syscall is an fd. There are also examples in which the "f" prefix does not even refer to an fd like in the case of "fork()", it is just part of the syscall name.

Syscalls' Naming Rule: What if a syscall's name starts with "l"?

I want to talk about those syscalls starting with "l". If we have a syscall "<syscall_name>" so we could also have "l<syscall_name>" which means that "l<syscall_name>" does the same operation as "<syscall_name>" but in case of a symbolic link given as input the information is retrieved about the link itself and not the file that the link refers to (for example "getxattr" and "lgetxattr". Moreover, not every syscall that starts with "l" falls in this category (think about "listen").

I think the last rule is the most confusing one because there are cases in which the "l" prefix is not part of the original name of the syscall and is not relevant to any type of links. Let us look at "lseek", the reason for having the prefix is to emphasize that the offset is given as long as opposed to the old "seek" syscall.

RCU (Read Copy Update)

Because there are multiple kernel threads (check it out using `ps -ef | grep rcu` - the output of the command is included in the screenshot at the end of the post) which are based on RCU (and other parts of the kernel) . I have decided to write a short explanation about it.

RCU is a sync mechanism which avoids the use of locking primitives in case multiple execution flows that read and write specific elements. Those elements are most of the times linked by pointers and are part of a specific data structure such as: has tables, binary trees, linked lists and more.

The main idea behind RCU is to break down the update phase into two different steps: “reclamation” and “removal” - let’s detail those phases. In the “removal” phase we remove/unlink/delete a reference to an element in a data structure (can be also in case of replacing an element with a new one). That phase can be done concurrently with other readers. It is safe due to the fact that modern CPUs ensure that readers will see the new or the old data but not partially updated. In the “reclamation” step the goal is to free the element from the data structure during the removal process. Because of that this step can disrupt a reader which references that specific element. Thus, this step must start only after all readers don’t hold a reference to the element we want to remove.

Due to the nature of the two steps an updater can finish the “removal” step immediately and defer the “reclamation” for the time all the active during this phase will complete (it can be done in various ways such as blocking or registering a callback).

RCU is used in cases where read performance is crucial but can bear the tradeoff of using more memory/space. Let’s go over a sequence of an update to a data structure in place using RCU. First, we just create a new data structure. Second, we copy the old data structure into the new one (don’t forget to save the pointer to the old data structure). Third, alter the new/copied data structure. Fourth, update the global pointer to reference the new data structure. Fifth, sleep until the kernel is sure they are no more readers using the old data structure (called also grace period, in Linux we can use `synchronize_rcu()`¹³.

In summary, RCU is probably the most common “lock-free” technique for shared data structures. It is lock-free for any number of readers. There are implementations also for single-writer and even multi-writers (However, it is out of scope for now). Of course, RCU also has problems and it is not designed for cases in which there are update-only scenarios (it is better for “mostly-read” and “few-writes”) - More about that in a future writeup.

¹³ <https://elixir.bootlin.com/linux/latest/source/kernel/rcu/tree.c#L3796>

root	3	2	0	07:15	?	00:00:00	[rcu_gp]
root	4	2	0	07:15	?	00:00:00	[rcu_par_gp]
root	10	2	0	07:15	?	00:00:00	[rcu_tasks_rude_]
root	11	2	0	07:15	?	00:00:00	[rcu_tasks_trace]
root	13	2	0	07:15	?	00:00:02	[rcu_sched]

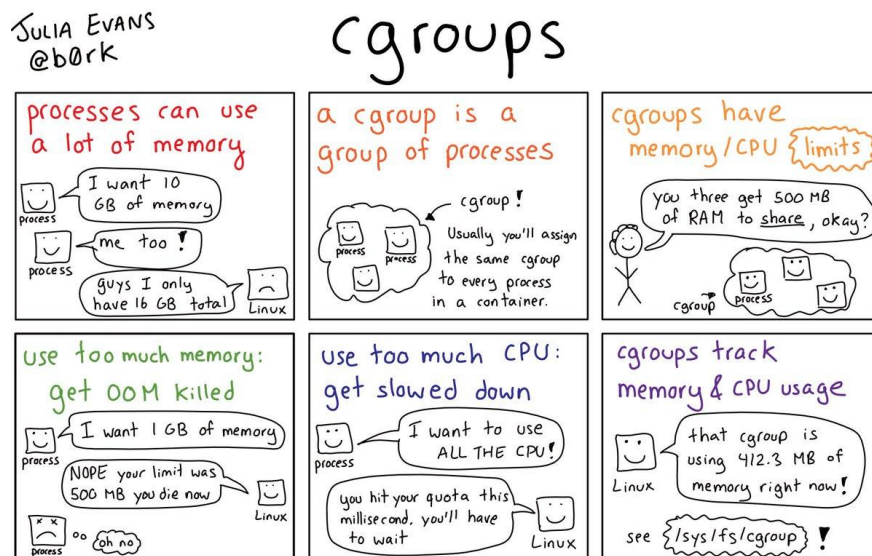
cgroups (Control Groups)

“Control Groups” (aka cgroups) is a Linux kernel feature that organizes processes into hierarchical groups. Based on those groups we can limit and monitor different types of OS resources. Among those resources are: disk I/O usage , network usage, memory usage, CPU usage and more (<https://man7.org/linux/man-pages/man7/cgroups.7.html>). cgroups are one of the building blocks used for creating containers (which include other stuff like namespaces, capabilities and overlay filesystems).

The cgroups functionality has been merged into the Linux kernel since version 2.6.24 (released in January 2008). Overall, cgroups provide the following features: resource limiting (as explained above), prioritization (some process groups can have larger shares of resources), control (freezing group of processes) and accounting¹⁴.

Moreover, there are two versions of cgroups. cgroups v1 was created by Paul Menage and Rohit Seth. cgroups v2 was redesigned and rewritten by Tejun Heo¹⁵. The documentation for cgroups v2 first appeared in the Linux kernel 4.5 release on March 2016¹⁶.

I will write on the differences between the two versions and how to use them in the upcoming writeups. A nice explanation regarding the concept of cgroups is shown in the image below¹⁷. By the way, since kernel 4.19 OOM killer¹⁸ is aware of cgroups, which means the OS can kill a cgroup as a single unit.



¹⁴ <https://docs.kernel.org/admin-guide/cgroup-v1/cgroups.html>

¹⁵ <https://www.wikiwand.com/en/Cgroups>

¹⁶ <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/diff/Documentation/cgroup-v2.txt?id=v4.5&id2=v4.4>

¹⁷ <https://twitter.com/b0rk/status/1214341831049252870>

¹⁸ <https://medium.com/@boutnaru/linux-out-of-memory-killer-oom-killer-bb2523da15fc>

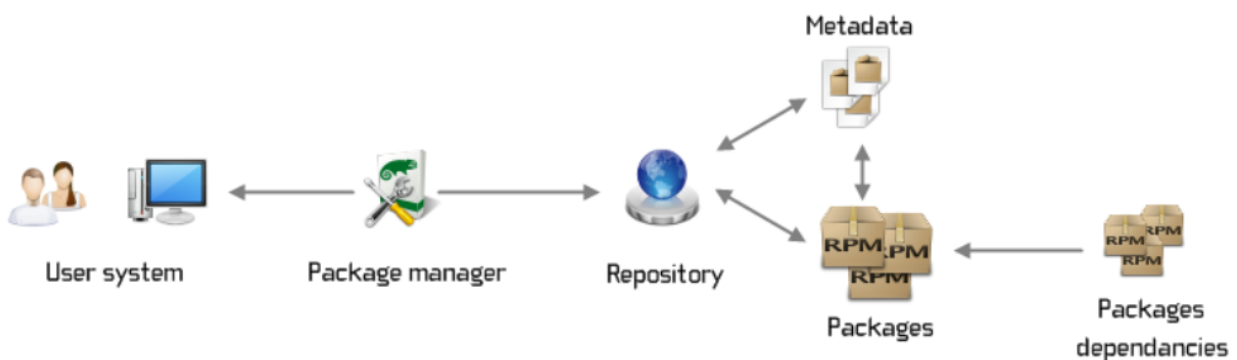
Package Managers

Package manager (aka “Package Management System”) is a set of software components which are responsible for tracking what software artifacts (executables, scripts, shared libraries and more). Packages are defined as a bundle of software artifacts that can be installed/removed as a group (<https://www.debian.org/doc/manuals/aptitude/pr01s02.en.html>).

Thus, we can say that a package manager automates the installation/upgrading/removing of computer programs from a system in a consistent manner. Moreover, package managers often manage a database that includes the dependencies between the software artifacts and version information in order to avoid conflicts (https://en.wikipedia.org/wiki/Package_manager).

Basically, there are different categories of package managers. The most common are: OS package managers (like dpkg, apk, rpm, dnf, pacman and more - part are only frontends as we will describe in the future) and runtime package managers focused on specific programming languages (like maven, npm, PyPi, NuGet, Composer and more). Each package manager can also have its own package file format (more on those in future writeups). Moreover, package managers can have different front-ends CLI based or GUI based. Those package managers can also support downloading software artifacts from different repositories (<https://devopedia.org/package-manager>).

Overall, package managers can store different metadata for each package like: list of files, version, authorship, licenses, targeted architecture and checksums. An overview of the package management flow is shown in the diagram below (<https://developerexperience.io/articles/package-management>).



What is an ELF (Executable and Linkable Format) ?

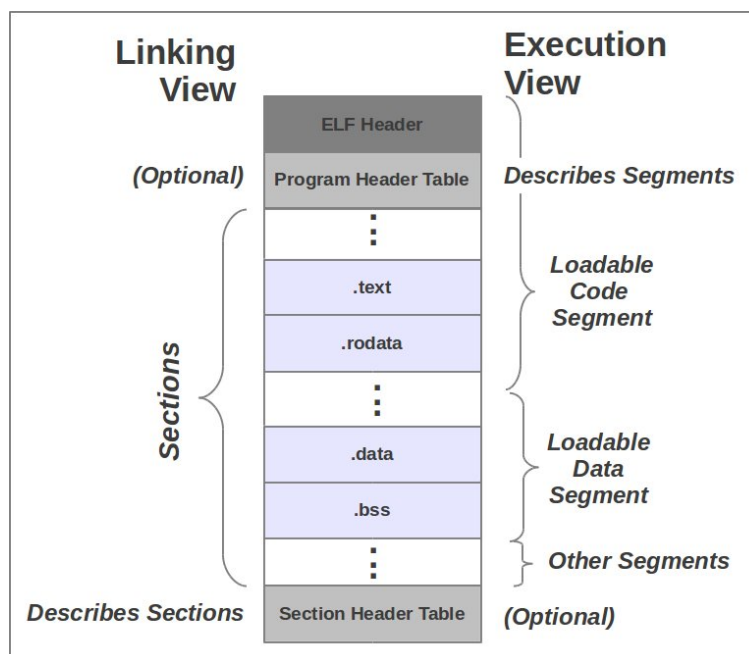
Every generic/standard operating system has a binary format for its user mode executables/libraries, kernel code and more. Windows has PE (Portable Executable), OSX has MachO and Linux has ELF. We are going to start with ELF (I promise to go over the others also).

In Linux ELF among the others (but not limited to) for executables, kernel models, shared libraries, core dumps and object files. Although, Linux does not mandates an extension for files ELF files may have an extension of *.bin, *.elf, *.ko, *.so, *.mod, *.o, *.bin and more (it could also be without an extension).

Moreover, today ELF is a common executable format for a variety of operating systems (and not only Linux) like: QNX, Android, VxWorks, OpenBSD, NetBSD, FreeBSD, Fuchsia, BeOS. Also, it is used in different platforms such as: Wii, Dreamcast and Playstation Portable.

ELF, might include 3 types of headers: ELF header (which is mandatory), program headers and sections header . The appearance of the last two is based on the goal of the file: Is it for linking only? Is it execution only? Both? (More on the difference between the two in the next chapters). You can see the different layouts of ELF in the image below¹⁹.

In the next parts we will go over each header in more detail. By the way, a great source for more information about ELF is man ("man 5 elf").



¹⁹ <https://i.stack.imgur.com/RMV0g.png>

The ELF (Executable and Linkable Format) Header

Now we are going to start with the ELF header. Total size of the header is 32 bytes. The header starts with the magic “ELF” (0x7f 0x45 0x4c 0x46).

From the information contained in the header we can answer the following questions: Is the file 32 or 64 bit? Does the file store data in big or little endian? What is the ELF version? The type of the file (executable, relocatable, shared library, etc)? What is the target CPU? What is the address of the entry point? What is the size of the other headers (program/section)? - and more.

If we want to parse the header of a specific ELF file we can use the command “readelf” (we are going to use it across all of the next parts to parse ELF files). In order to show the header of an ELF file we can run “readelf -h {PATH_TO_ELF_FILE}”. In the image below we can see the ELF header of “ls”. The image was taken from an online Arch Linux in a browser (copy.sh).

```
root@localhost:~# readelf -h `which ls`
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF32
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  DYN (Position-Independent Executable file)
  Machine:                               Intel 80386
  Version:                               0x1
  Entry point address:                   0x3c60
  Start of program headers:              52 (bytes into file)
  Start of section headers:              156320 (bytes into file)
  Flags:                                  0x0
  Size of this header:                    52 (bytes)
  Size of program headers:                32 (bytes)
  Number of program headers:              12
  Size of section headers:                40 (bytes)
  Number of section headers:              28
  Section header string table index:      27
```

File System Hierarchy in Linux

As it turns out, there is a standard which is a reference that describes the conventions used by Unix/Linux systems for the organization and layout of their filesystem. This standard was created about 28 years ago (14 Feb 1994) and the last version (3.0) was published 7 years ago (3 Jun 2015). If you want to go over the specification for more details use the following link - <https://refspecs.linuxfoundation.org/fhs.shtml>.

We are going to give a short description for each directory (a detailed description for some of them will be in a dedicated write-up). We are going to list all the directories based on a lexicographic order. All the examples that I am going to share are based on a VM running Ubuntu 22.04.1 (below is a screenshot showing the directories for that VM). So let the fun begin ;-)

“/”, is the root directory of the entire system (the start of everything).

“/bin”, basic command mostly binaries (there are also scripts like `zgrep`) that are needed for every user. Examples are: `ls`, `ip` and `id`.

“/boot”, contains files needed for boot like the kernel (`vmlinuz`), `initrd` and boot loader configuration (like for `grub`). It may also contain metadata information about the kernel build process like the config that was used (A detailed writeup is going to be shared about “/boot” in the future).

“/dev”, device files, for now you should think about it as an interface to a device driver which is located on a filesystem (more on that in the future). Examples are: `/dev/null`, `/dev/zero` and `/dev/random`.

“/etc”, contains configuration about the system or an installed application. Examples are: `/etc/adduser.conf` (configuration file for `adduser` and `addgroup` commands) and `/etc/sudo.conf`.

“/home”, is the default location of the users’ home directory (it can be modified in `/etc/passwd` per user). The directory might contain personal settings of the user, saved files by the user and more. Example is `.bash_history` which is a hidden file that contains the historical commands entered by the user (while using the bash shell).

“/lib”, contains libraries needed by binaries mostly (but not limited to) in “/bin” and “/sbin”. On 64 bit systems we can also have “lib64”.

“/media”, used as a mount point for removable media (like CD-ROMs and USBs).

“/mnt”, can be used for temporary mounted filesystems.

“/opt”, should include applications installed by the user as add-ons (in reality not all of the addons are installed there).

“/lost+found”, this directory holds files that have been deleted or lost during a file operation. It means that we have an inode for those files, however we don’t have a filename on disk for them (think about cases of kernel panic or an unplanned shutdown). It is handled by tools like `fsck` - more on that is a future writeup.

“/proc”, it is a pseudo filesystem which enables retrieval of information about kernel data structures from user space using file operations, for example “ps” reads information for there to build the process list. Due to the fact it is a crucial part of Linux I am going to dedicate an entire writeup about it.

“/root”, it's the default home directory of the root account.

“/run”, it is used for runtime data like: running daemons, logged users and more. It should be erased/initialized every time on reboot/boot.

“/sbin”, similar to “/bin” but contains system binaries like: lsmod, init (in Ubuntu by the way it is a link to systemd) and dhclient.

“/srv”, contains information which is published by the system to the outside world using FTP/web server/other.

“/sys”, also a pseudo filesystem (similar to /proc) which exports information about hardware devices, device drivers and kernel subsystems. It can also allow configuration of different subsystems (like tracing for ftrace). I will cover it separately in more detail in the near future.

“/tmp”, the goal of the directory is to contain temporary files. Most of the time the content is not saved between reboots. Remember that there is also “/var/tmp”.

“/usr”, it is referred to by multiple names “User Programs” or “User System Resources”. It has several subdirectories containing binaries, libs, doc files and also can contain source code. Historically, it was meant to be read-only and shared between FHS-compliant hosts (<https://tldp.org/LDP/Linux-Filesystem-Hierarchy/html/usr.html>). Due to the nature of its complexity today and the large amount of files it contains we will go over it also in a different writeup.

“/var”, aka variable files. It contains files which by design are going to change during the normal operation of the system (think about spool files, logs and more). More on this directory in the future.

It is important to note that those are not all the directories and subdirectories included on a clean Linux installation, but the major ones I have decided to start with (more information will be shared in the future). See you soon ;-)

```
drwxr-xr-x  3 root root      4096 Sep 16 22:02 boot
drwxr-xr-x 19 root root      4120 Sep 18 05:47 dev
drwxr-xr-x 135 root root    12288 Sep 16 22:02 etc
drwxr-xr-x  3 root root      4096 Jul 29 07:13 home
drwxrwxrwx  1 root root         7 Apr 19 06:02 lib -> usr/lib
drwxrwxrwx  1 root root         9 Apr 19 06:02 lib32 -> usr/lib32
drwxrwxrwx  1 root root         9 Apr 19 06:02 lib64 -> usr/lib64
drwxrwxrwx  1 root root        10 Apr 19 06:02 libx32 -> usr/libx32
drwx-----  2 root root    16384 Jul 29 07:10 lost+found
drwxr-xr-x  2 root root      4096 Apr 19 06:02 media
drwxr-xr-x  2 root root      4096 Apr 19 06:02 mnt
drwxr-xr-x  2 root root      4096 Apr 19 06:02 opt
dr-xr-xr-x 273 root root         0 Sep  9 12:31 proc
drwx-----  6 root root      4096 Sep 12 18:40 root
drwxr-xr-x 29 root root      840 Sep 18 00:54 run
drwxr-xr-x  9 root root      4096 Apr 19 06:08 snap
drwxr-xr-x  2 root root      4096 Apr 19 06:02 srv
dr-xr-xr-x 13 root root         0 Sep  9 12:31 sys
drwxrwxrwt 19 root root      4096 Sep 18 07:20 tmp
drwxr-xr-x 14 root root      4096 Apr 19 06:02 usr
drwxr-xr-x 14 root root      4096 Apr 19 06:07 var
```

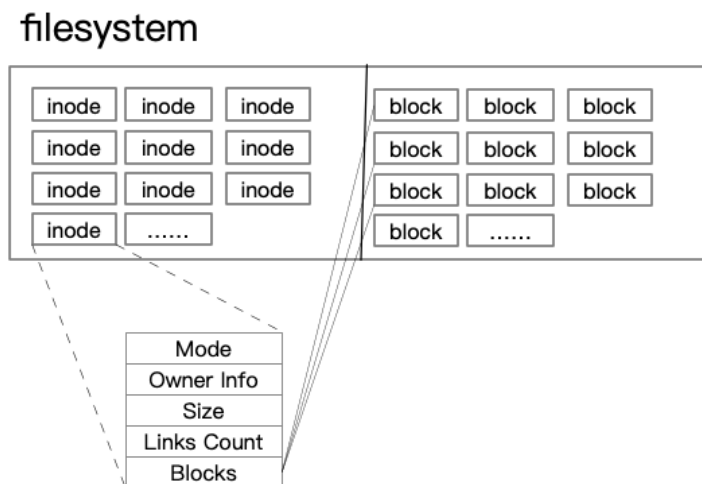
What is an inode?

An inode (aka index node) is a data structure used by Unix/Linux like filesystems in order to describe a filesystem object. Such an object could be a file or a directory. Every inode stores pointers to the disk blocks locations of the object's data and metadata²⁰. An illustration of that is shown below²¹.

Overall, the metadata contained in an inode is: file type (regular file/directory/symbolic link/block special file/character special file/etc), permissions, owner id, group id, size, last accessed time, last modified time, change time and number of hard links²².

By using inodes the filesystem tracks all files/directories saved on disk. Also, by using inodes we can read any specific byte in the data of a file very effectively. We can see the number of total inodes per mounted filesystem using the command “df -i”²³. Also, we can see the inode of a file/directory and other metadata of the file using the command “ls -li”²⁴ or “stat”²⁵. By the way, the “stat” command can use different syscalls (depending on the filesystem and the specific version) like “stat”²⁶, “lstat”²⁷ or “statx”²⁸.

Lastly, you can check out “struct inode” in the source code of the Linux kernel²⁹. Not all the points/links are directly connected to the data blocks, however I will elaborate on that in a future writeup.



²⁰ <https://www.bluematador.com/blog/what-is-an-inode-and-what-are-they-used-for>

²¹ <https://www.sobyte.net/post/2022-05/linux-inode/>

²² <https://www.stackscale.com/blog/inodes-linux/>

²³ <https://linux.die.net/man/1/df>

²⁴ <https://man7.org/linux/man-pages/man1/ls.1.html>

²⁵ <https://linux.die.net/man/1/stat>

²⁶ <https://linux.die.net/man/2/stat>

²⁷ <https://linux.die.net/man/2/lstat>

²⁸ <https://man7.org/linux/man-pages/man2/statx.2.html>

²⁹ <https://elixir.bootlin.com/linux/v6.4.2/source/include/linux/fs.h#L612>

Why is removing a file not dependent on the file's permissions?

Something which is not always understood correctly by Linux users is the fact that removing a file is not dependent on the permissions of the file itself. As you can see in the screenshot below even if a user has full permission (read+write+execute) it can't remove a file. By the way, removing a file is done by using the "unlink" syscall³⁰ or the "unlinkat" syscall³¹.

The reason for that is because the data that states a file belongs to a directory is saved as part of the directory itself. We can think about a directory as a "special file" whose data is the name and the inode³² numbers of the files that are part of that specific directory.

Thus, if we add write permissions to the directory even if the user has no permissions to the file ("chmod 000") the file can be removed (from the directory) - as shown in the screenshot below.

```
[troller@localhost test]$ ls -lah ./troller
-rwxrwxrwx 1 root root 8 Jul 11 2023 ./troller
[troller@localhost test]$ cat ./troller
Troller
[troller@localhost test]$ rm ./troller
rm: cannot remove './troller': Permission denied
[troller@localhost test]$ exit
exit
root@localhost:/tmp/test# chmod o+w /tmp/test
root@localhost:/tmp/test# chmod 000 ./troller
root@localhost:/tmp/test# su troller
[troller@localhost test]$ ls -lah ./troller
----- 1 root root 8 Jul 11 2023 ./troller
[troller@localhost test]$ cat ./troller
cat: ./troller: Permission denied
[troller@localhost test]$ rm ./troller
rm: remove write-protected regular file './troller'? y
[troller@localhost test]$ ls -lah ./troller
ls: cannot access './troller': No such file or directory
```

³⁰ <https://linux.die.net/man/2/unlink>

³¹ <https://linux.die.net/man/2/unlinkat>

³² <https://medium.com/@boutnaru/linux-what-is-an-inode-7ba47a519940>

VFS (Virtual File System)

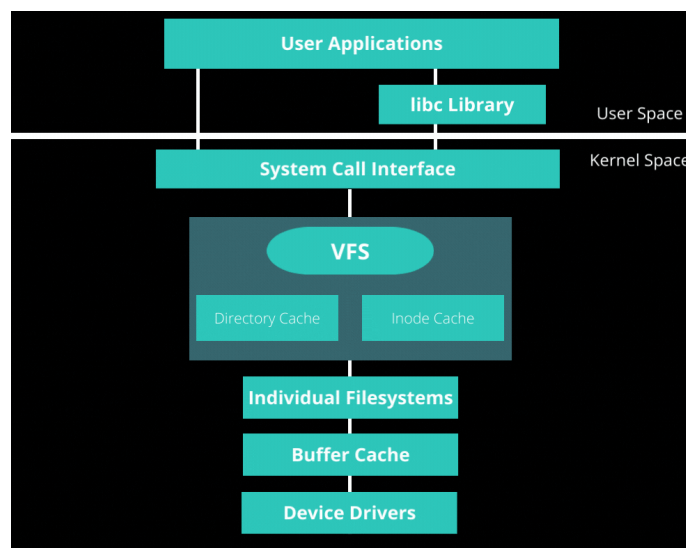
VFS (Virtual File System, aka Virtual File Switch) is a software component of Linux which is responsible for the filesystem interface between the user-mode and kernel mode. Using it allows the kernel to provide an abstraction layer that makes implementation of different filesystems very easy³³.

Overall, VFS is masking the implementation details of a specific filesystem behind generic system calls (open/read/write/close/etc), which are mostly exposed to user-mode application by some wrappers in libc - as shown in the diagram below³⁴.

Moreover, we can say that the main goal of VFS is to allow user-mode applications to access different filesystems (think about NTFS, FAT, etc.) in the same way. There are four main objects in VFS: superblock, dentries, inodes and files³⁵.

Thus, “inode”³⁶ is what the kernel uses to keep track of files. Because a file can have several names there are “dentries” (“directory entries”) which represent pathnames. Also, due to the fact a couple of processes can have the same file opened (for read/write) there is a “file” structure that holds the information for each one (such as the cursor position). The “superblock” structure holds data which is needed for performing actions on the filesystem - more details about all of those and more (like mounting) are going to be published in the near future.

Lastly, there are also other relevant data structures that I will post on in the near future (“filesystem”, “vfsmount”, “nameidata” and “address_space”).



³³ <https://www.kernel.org/doc/html/next/filesystems/vfs.html>

³⁴ <https://www.starlab.io/blog/introduction-to-the-linux-virtual-filesystem-vfs-part-i-a-high-level-tour>

³⁵ <https://www.win.tue.nl/~aeb/linux/lk/lk-8.html>

³⁶ <https://medium.com/@boutnaru/linux-what-is-an-inode-7ba47a519940>